Рабочая тетрадь студента

Кружок: «Кодим под Android»
Тема занятия: Алгоритмы и блок-схемы
Продолжительность: 4 академических часа
Уровень: начинающий
ФИО студента: Коршунова
Группа: 1рки-4-26
Дата: 11.10

Цель занятия

Научиться составлять алгоритмы и записывать их в виде блок-схем, чтобы в дальнейшем легко переводить их в код на Kotlin.

Что ты узнаешь сегодня

Как описывать порядок действий словами.
Как рисовать блок-схемы для разных видов алгоритмов.
Как проверять алгоритм на ошибки.

Зачем это нужно

Программист сначала **продумывает алгоритм**, а потом пишет код. Если алгоритм неправильный — программа не заработает, даже если код написан без ошибок. Блок-схема помогает **увидеть логику** до того, как ты начнёшь писать код. На собеседованиях в IT-компаниях часто просят **объяснить алгоритм** решения задачи — без этого навыка не обойтись.

Часть 1. Теория (2 часа)

1.1. Алгоритм — это порядок действий

Алгоритм — это чёткая последовательность шагов для решения задачи.

Зачем это нужно: компьютер не умеет думать сам. Ему нужно точно сказать, что делать и в каком порядке. Если порядок нарушить — результат будет неправильным.

Пример: «Как открыть дверь ключом»

1. Достать ключ.
2. Вставить ключ в замок.
3. Повернуть ключ.
4. Нажать на ручку.
5. Открыть дверь.

Разбор: если поменять шаги 1 и 2 местами — вставить ключ, которого нет в руке, — не получится. Порядок важен.

1.2. Свойства алгоритма

Дискретность — состоит из отдельных шагов.
Понятность — каждый шаг ясен исполнителю.
Конечность — заканчивается за конечное число шагов.
Результативность — приводит к результату.
Массовость — подходит для похожих задач.

Зачем это нужно: если алгоритм непонятен, бесконечен или не даёт результата — это не алгоритм, а набор бесполезных действий. Проверяй свои алгоритмы по этим свойствам.

Пример нарушения понятности: «Сделай красиво» — непонятный шаг, потому что «красиво» каждый понимает по-своему.

1.3. Виды алгоритмов

Линейный — все шаги выполняются один раз по порядку.
Ветвящийся — есть выбор из двух вариантов по условию.
Циклический — некоторые шаги повторяются.
Комбинированный — сочетает несколько видов.

Зачем это нужно: в программировании эти виды превращаются в конкретные конструкции языка: линейный — в последовательные команды, ветвящийся — в `if/when`, циклический — в `for/while`. Поняв вид алгоритма, ты сразу знаешь, какую конструкцию использовать в коде.

1.4. Три способа записи алгоритма

Словесный — обычными предложениями.
Графический — блок-схемой.
Программный — на языке программирования.

Зачем это нужно: на занятии ты будешь использовать все три. Сначала — словами, потом — блок-схемой, потом — кодом. Так ты научишься переводить мысль в программу.

Пример одной и той же задачи тремя способами:

Словесно: «Если число больше нуля — вывести "плюс", иначе — "минус"».
Блок-схемой: ромб с условием и две ветки.
Программно:

```kotlin
if (x > 0) {
    println("плюс")
} else {
    println("минус")
}
```

1.5. Элементы блок-схемы

Овал — начало или конец алгоритма.
Прямоугольник — действие.
Ромб — условие с двумя выходами: да и нет.
Параллелограмм — ввод или вывод данных.
Стрелка — показывает направление.

Зачем это нужно: блок-схема — это универсальный язык. Её поймёт программист на любом языке. Если ты умеешь рисовать блок-схему — ты сможешь объяснить свою идею кому угодно.

1.6. Как читать блок-схему

Движение всегда начинается с овала «Начало» и идёт по стрелкам.
В ромбе задаётся вопрос — от него два пути: «да» и «нет».
Овал «Конец» завершает алгоритм.

Зачем это нужно: читать чужой код сложно, а читать чужую блок-схему — легко. Это навык работы в команде.

1.7. Как проверить алгоритм

Тестирование — прогон на разных данных.
Трассировка — пошаговое выполнение с записью значений.
Проверка границ — поведение на крайних значениях.

Зачем это нужно: программист всегда проверяет свою программу. Если не проверить алгоритм на бумаге — придётся искать ошибку в коде, а это в разы дольше.

Пример проверки границ: если алгоритм проверяет «возраст ≥ 18», обязательно проверь значения 17, 18 и 19 — именно на границах чаще всего бывают ошибки.

Проверь себя (устно)

Что означает ромб в блок-схеме? Условие да/нет
Сколько выходов у условия? Два – да и нет
Чем циклический алгоритм отличается от линейного? В циклическом алгоритме некоторые шаги повторяются, а в линейном все шаги выполняются один раз по порядку
Зачем проверять алгоритм на граничных значениях? Чтобы убедиться, что на крайних значениях алгоритм работает правильно и не дает сбоев

Часть 2. Практика (2 часа)

Задание 1. Составь словесный алгоритм (10 минут)

Зачем: научиться формулировать шаги чётко, чтобы их понял любой исполнитель.

Составь алгоритм **«Как помыть посуду»**. Запиши 5–6 шагов.

Подсказка: начни с «взять губку» и не забудь про сушку.

Пример начала: 1. Открыть кран. 2. ...

1. Включить воду
2. Взять губку и налить мыло на нее
3. Помыть посуду с мылом
4. Смыть мыло
5. Вытереть посуду
6. Положить на место

Вид алгоритма: Линейный

Задание 2. Определи вид (10 минут)

Зачем: быстро распознавать вид алгоритма, чтобы выбрать правильную конструкцию в коде.

А) «Включить компьютер, ввести пароль, нажать Enter».
Ответ: Линейный, потому что шаги выполняются один раз по порядку

Б) «Если есть домашнее задание — сделать, иначе — отдохнуть».
Ответ: ветвящийся, потому что есть условие и варианты действий

В) «Повторять 5 раз: присесть и встать».
Ответ: циклический, потому что действие повторяется несколько раз

Г) «Пока не выучишь стих, повторяй: прочитай, расскажи, проверь».
Ответ: циклический, потому что есть повторяющиеся действия

Пример ответа на А: линейный, потому что шаги выполняются один раз по порядку.

Задание 3. Построй блок-схему с условием (20 минут)

Зачем: научиться переводить условие «если..., иначе...» в графическую форму — это основа будущего `if/when` в Kotlin.

Алгоритм «Проверка пароля»:

Если пароль верный — вывести «Добро пожаловать».
Иначе — вывести «Пароль неверный».

Нарисуй блок-схему:

(Начало)
[↓Ввод пароля]

<↓Пароль верный?>

1)Да

[Добро пожаловать]

↓2) Нет

[Пароль неверный]

↓
(Конец)

Подсказка: используй параллелограмм для ввода пароля, ромб для проверки и два прямоугольника для вывода.

Задание 4. Построй блок-схему с циклом (20 минут)

Зачем: циклы в коде (`for`, `while`) — одни из самых сложных тем для новичков. Блок-схема помогает понять, что именно повторяется и когда цикл заканчивается.

Алгоритм «Налить 3 чашки чая»:

Пока чашек меньше 3 — налить чашку и увеличить счётчик.

Нарисуй блок-схему:

(Начало)
↓[Налитых чашек = 0]

↓<Налитых кружек = 3?>

↓Нет

[Налить чашку]

↓Да

↓
(Конец)

Подсказка: начало счётчика — прямоугольник, проверка условия — ромб, тело цикла — прямоугольник, возврат к проверке — стрелка вверх.

Задание 5. Трассировка (15 минут)

Зачем: уметь вручную «прогнать» алгоритм и получить результат. Этот навык нужен для отладки кода.

Дан алгоритм:

1. X = 7.
2. Если X > 5 — вывести «Много».
3. Иначе — вывести «Мало».
4. Конец.

Что выведет алгоритм? Много

А если X = 5? Мало

А если X = 1? Мало

Пример решения для X = 7: условие 7 > 5 истинно, значит выводится «Много».
Подсказка для X = 5: проверь, строгое ли неравенство.

Задание 6. Найди ошибку (15 минут)

Зачем: в программировании ошибки в порядке шагов — самая частая причина, почему «ничего не работает». Учись видеть их сразу.

Алгоритм «Приготовить бутерброд»:

1. Съесть бутерброд.
2. Взять хлеб.
3. Намазать маслом.
4. Положить сыр.

Что не так? Неправильный порядок действий

Исправленный порядок:

1. Взять хлеб

---

2. Намазать маслом

---

3. Положить сыр

---

4. Съесть бутерброд

Подсказка: нельзя съесть то, что ещё не приготовлено.

Задание 7. Мини-проект (25 минут)

Зачем: собрать все навыки вместе — придумать алгоритм, определить вид, нарисовать блок-схему. Это то, что делает программист перед тем, как сесть за код.

Придумай свой алгоритм для задачи по выбору.
Например: «Как найти книгу в библиотеке», «Как записаться в кружок», «Как вызвать такси».

Название алгоритма: Как вызвать такси

Шаги:

1. Взять телефон
2. Открыть приложение такси
3. Выбрать тариф
4. Указать адрес откуда и куда надо приехать
5. Дождаться такси и сесть в машину, когда таксист приедет

Вид алгоритма: Линейный

Блок-схема:

(Начало)
[↓Взять телефон]

[↓Открыть приложение такси]

[↓Выбрать тариф]

[↓Указать адрес откуда и куда надо приехать]

[↓Дождаться такси и сесть в машину, когда таксист приедет]
(Конец)

Итог занятия

Что я узнал(а):

Как описывать порядок действий словами
Как делать блок-схемы для алгоритмов
Как проверять алгоритм на ошибки

Что осталось непонятным:

Все понятно

Что понравилось:

Делать блок-схемы

Домашнее задание

1. Составь алгоритм «Как добраться до колледжа» (5–6 шагов) и определи его вид.

1) Выйти из дома

2)Дойти до метро

3)Зайти в метро

4)Дождаться поезда и сесть в него

5)Доехать до нужной станции

6)Выйти из метро

7)Дойти до колледжа

2. Нарисуй блок-схему для алгоритма «Проверка оценки».

(Начало)

↓

[Ввод оценки]

↓

<Оценка >= 4>

↓ Нет

[Надо исправить]

↓

Да

[Спасибо хорошо]

↓

(Конец)

3. Придумай циклический алгоритм из повседневной жизни и нарисуй его блок-схему.

(Начало)

↓

[Встать с кровати]

↓

[Зайти на кухню]

↓

[Взять губку в одну руку, во вторую – посуду]

↓

[Помыть посуду]

↓

[Смыть мыло с посуды]

↓

[Протереть посуду]

↓

[Положить на место]

↓

[Вытереть руки]

↓

[Уйти из кухни]

↓

(Конец)

Оценка за занятие: _____

Подпись педагога: _____

Памятка. Элементы блок-схемы

Овал — начало или конец.
Прямоугольник — действие.
Ромб — условие.
Параллелограмм — ввод или вывод.
Стрелка — направление.

Важно:

Движение — только по стрелкам.
У ромба всегда два выхода: да и нет.
Цикл завершается, когда условие становится ложным.
Алгоритм заканчивается овалом «Конец».

Рабочая тетрадь разработана для кружка «Кодим под Android»
ГБПОУ «ИТ.Москва», 2026 г.